Software Requirements Specification (SRS)

Revision history

Version	Change summary
1.0	Project synopsis (Phase 1)
2.0	Full SRS, validation specification and SPMP
2.1	Moved to course SRS template. Added acceptance criteria and test references for every FR, security objectives, two use case diagrams and RTM status column.
2.2	Updated to the system as built at the end of Sprint 2. Holds, waitlist, check-ins and audit now in PostgreSQL; email replaced by in-app notifications; staff accounts provisioned by admins; RTM statuses and coverage figures updated; deviations referenced. No requirement removed.

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes and acceptance tests
7. System models and diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document specifies the software requirements for PassPulse version 2.2, a web platform for event discovery, multi-tier ticket booking, signed QR check-in and live attendance tracking. It defines functional and non-functional requirements, external interfaces and the acceptance criteria used to verify each requirement. It is the baseline for the Software Architecture and Design Specification (SAD) and the Software Test Plan (STP). Version 2.2 reflects the system as built at the end of Sprint 2; where the build differs from version 2.1 the change is noted against the requirement and logged in `docs/DEVIATIONS.md`.

1.2 Scope

In scope: user registration and role-based access for four roles; event creation, approval and lifecycle; multi-tier ticket quotas; booking with a 10-minute inventory hold; HMAC-signed QR passes; camera-based gate check-in with duplicate rejection; a FIFO waitlist with a 15-minute claim window and in-app notifications; a live organizer dashboard; CSV/PDF reports; and admin moderation.

Out of scope: real payment gateway integration (checkout is a mock confirmation step), native Android/iOS apps (the web client is mobile responsive), RFID or turnstile hardware, live streaming, facial recognition and ERP/accounting integration.

1.3 Audience

Course instructors and evaluators, the development team, peer reviewers from other teams, and team members acting as testers during the validation phase.

1.4 Definitions

Term	Meaning
RBAC	Role-Based Access Control. Access to routes and actions decided by the user's role.

Term	Meaning
JWT	JSON Web Token (RFC 7519). Signed bearer token used for stateless authentication.
TTL	Time-To-Live. Automatic key expiry in Redis, used for the 600-second ticket hold.
HMAC-SHA256	Keyed hash used to sign QR pass payloads so that edited passes are detected.
FIFO	First-In, First-Out ordering of the waitlist.
Hold	A temporary reservation of ticket inventory for one user during checkout.
Pass	The signed QR ticket issued for one seat of a confirmed booking.
Scan velocity	Number of successful check-ins per minute at the gates of an event.
SPA, API, SSE, TLS, CI	Single Page Application, Application Programming Interface, Server-Sent Events, Transport Layer Security, Continuous Integration.

2. Overall description

2.1 Product perspective

PassPulse is a new, self-contained web application. It replaces the mix of Google Forms, static PDF tickets and spreadsheet roll-calls that student clubs use today. The client is a React single-page application. The server is a Node.js and Express REST API. PostgreSQL stores bookings, tickets, holds, the waitlist, check-ins, notifications and the audit log. Redis holds a short-lived projection of each active hold with an absolute expiry time. Users and events are currently served from an in-memory repository mirrored into PostgreSQL when a booking needs them (DEV-002, DEV-006). Waitlist offers are delivered as in-app notifications instead of email (DEV-003). An Nginx reverse proxy terminates TLS 1.3 in front of the API and client. The whole stack runs in Docker Compose.

2.2 Major product functions

- Register, log in and enforce role permissions (Attendee, Organizer, Gate Staff, Admin)
- Search and filter published events by keyword, category, date, venue and price
- Create events, configure ticket tiers and move events through their lifecycle states
- Hold tickets for 10 minutes and confirm bookings without overselling
- Issue HMAC-signed, time-limited QR passes for confirmed tickets
- Scan passes at the gate and reject invalid, expired or duplicate passes
- Run a FIFO waitlist and offer released seats with a 15-minute claim window
- Show live check-in telemetry and export attendance and revenue reports
- Let admins approve or flag events, deactivate accounts and review audit logs

2.3 User roles and characteristics

Role	Characteristics	Main needs
Attendee	Students and the general public. Non-technical, mostly on phones. Arrive in bursts when a popular event opens for booking.	Fast search, a clear hold timer, a pass that works offline once loaded.
Organizer	Club leads and event managers. Comfortable with forms, want numbers.	Simple event wizard, tier setup, live attendance and exports.
Gate Staff	Volunteers at the entrance, often with little training, working in a queue.	One-screen scanner with an unambiguous accept/reject result in under 200 ms.
Admin	Platform administrators with elevated rights.	Moderation queue, account controls, audit log.

2.4 Operating environment

- Server: Docker containers, Node.js 20.x, Express
- Data: PostgreSQL 16 and Redis 7; proxy: Nginx with TLS 1.3 (all run from `docker-compose.yml`)

- Client: current Chrome, Firefox, Edge and Safari on desktop and mobile; camera access through the HTML5 MediaDevices API on gate devices
- Network: gate devices need at least 1 Mbps; all server clocks synchronised by NTP

2.5 Constraints

- Inventory changes run inside PostgreSQL transactions with row locks and the check `sold + reserved <= quota`, so the platform never sells more seats than a tier holds. Redis mirrors active holds but is not the source of truth.
- All protected API calls use stateless JWT bearer tokens; no server-side sessions.
- Gate validation, including the signature check and the check-in write, must finish in under 200 ms.
- Server code must keep Jest statement, branch and line coverage at or above 80%, and the client must pass `oxlint` and `vite build`, in GitHub Actions before merge to `main`.
- Secrets (`JWT_SECRET`, `QR_HMAC_SECRET`, database credentials) come from environment variables and are never committed.
- Payment is simulated; no card data is collected or stored.

3. External interface requirements

3.1 User interfaces

- **Discovery and catalog:** grid of event cards with a search bar and filters for category, date range, venue and price.
- **Event wizard:** multi-step form for title, venue, date and time, description, banner image and ticket tiers.
- **Checkout and hold:** order summary with a visible 10:00 countdown and a confirm button. The catalog is the home page, so the journey is catalog, event, checkout, pass (NFR-005).
- **QR pass:** mobile view with attendee name, event, tier and the QR code at a size readable from 30 cm.
- **Gate scanner:** camera viewfinder; full-screen green result for a valid pass and red with the reason (duplicate, wrong event, expired, invalid) otherwise.
- **Organizer dashboard:** check-in progress bar, scan velocity chart, export buttons.
- **Notifications:** in-app list of waitlist offers with accept and decline actions.
- **Admin panel:** moderation queue, user list with deactivate and reactivate, audit log view.
- **Accessibility:** text contrast at least 4.5:1, all actions reachable by keyboard, result of a scan shown by text and colour, never colour alone.

3.2 Hardware interfaces

The only hardware dependency is the device camera on gate staff phones or laptops, accessed through `navigator.mediaDevices.getUserMedia` by the html5-qrcode library. No dedicated scanner hardware is required.

3.3 Software interfaces

- **PostgreSQL 16** through the `pg` connection pool with parameterised queries only.
- **Redis 7** for hold keys written with `SET hold:{id} NX PXAT`. A worker in the API releases expired holds from PostgreSQL every second and reconciles stale Redis keys.
- **No SMTP.** Waitlist offers are stored in the `notifications` table and shown in the client (DEV-003).
- **qrcode** npm package to render the signed payload as a PNG data URL.

3.4 Communications

- REST over HTTPS with JSON bodies. TLS 1.3 is terminated at the Nginx proxy, which redirects HTTP to HTTPS and sends HSTS; plain HTTP is allowed only on localhost during development.

- Authorization header: `Bearer <JWT>`, token lifetime 1 day (`JWT_EXPIRES_IN=1d`).
- Live dashboard updates use Server-Sent Events from `/api/telemetry/:eventId/stream`. Because EventSource cannot send headers, this route also accepts the token as a query parameter.
- Error responses follow one shape: `{ status: "error", code, message }`.

4. System features (detailed)

Each requirement has acceptance criteria and a reference to the test case in the STP. IDs follow FR-### and are unchanged from SRS v2.0. Notes marked "As built" describe how version 2.2 differs from the original wording. Priority is High, Medium or Low.

4.1 Authentication and RBAC

Description: registration, login and role checks on every protected route.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-001	The system shall let a user register with a unique email, password and full name.	Functional	High	All users	AC: valid input returns 201 and a user row; a repeated email returns 409 EMAIL_EXISTS. Test: TC-AUTH-01, TC-AUTH-02	As built: self-registration always creates an ATTENDEE. Organizer, gate staff and admin accounts are created by an admin through `POST /api/auth/provision` (TC-PROV-01 to 05).
FR-002	The system shall hash passwords with bcrypt (cost ≥ 10) before storing them.	Functional	High	Security	AC: stored value starts with `$2` and cost field ≥ 10; plaintext never in DB or logs. Test: TC-AUTH-03	bcryptjs
FR-003	The system shall authenticate requests with JWT bearer tokens and return 403 when the role is not allowed on the route.	Functional	High	Security	AC: missing or bad token gives 401; attendee token on an organizer route gives 403. Test: TC-AUTH-04 to TC-AUTH-09, TC-MID-01 to 05	Needed by every module below.

4.2 Event discovery and lifecycle

Description: attendees find events; organizers create and manage them through defined states.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-004	The system shall let users filter published events by keyword, category, date range, venue and ticket price.	Functional	High	Attendee	AC: each filter alone and combined returns only matching PUBLISHED or LIVE events; an empty result shows "No events found". Test: TC-EVT-01, TC-EVT-02	Indexed columns: category, event_date.
FR-005	The system shall let organizers create, edit, schedule and cancel their own events with title, venue, capacity and date/time.	Functional	High	Organizer	AC: event saved in DRAFT; an organizer cannot edit another organizer's event (403). Test: TC-EVT-03	BR-006 limits edits after bookings exist.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-006	The system shall keep event state as DRAFT, PENDING_APPROVAL, PUBLISHED, LIVE, COMPLETED or CANCELLED and allow only the transitions in SAD Figure B-3.	Functional	Medium	Organizer, Admin	AC: an illegal transition (e.g. DRAFT to LIVE) returns 409 INVALID_TRANSITION. Test: TC-EVT-04	BR-005 checked on submit.

4.3 Ticket tiers and booking engine

Description: tier quotas, the 10-minute hold and the atomic booking commit.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-007	The system shall let organizers define tiers (e.g. Early Bird, General, VIP) with their own quota and price.	Functional	High	Organizer	AC: tier with quota 0 or negative price rejected with 400. Test: TC-TCK-01	
FR-008	The system shall hold the selected tickets for exactly 10 minutes during checkout with a 600 s expiry held in PostgreSQL and mirrored to Redis.	Functional	High	Business	AC: hold response carries `expiresAt` = now + 600 s (±1 s); held seats are not offered to others. Test: TC-BKG-01	As built: `ticket_holds` row plus `reserved` counter; Redis key uses PXAT.
FR-009	The system shall return held seats to stock when the hold expires without confirmation.	Functional	High	Business	AC: after 600 s the tier availability is back to its earlier value. Test: TC-BKG-02	As built: hold expiry worker every 1 s with `FOR UPDATE SKIP LOCKED`, replacing keyspace notifications.
FR-010	The system shall commit confirmed bookings inside one PostgreSQL transaction so that sold seats never exceed the tier quota.	Functional	High	Business	AC: 200 concurrent confirms on a 1-seat tier produce exactly 1 booking. Test: TC-BKG-03, TC-PERF-03	Confirm via `POST /api/bookings/confirm`; booking, tickets and audit row in one transaction.

4.4 QR pass generation

Description: one signed, expiring pass per confirmed ticket.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-011	The system shall generate a QR pass for each confirmed ticket containing ticketId, eventId, userId, issuedAt, expiresAt and an HMAC-SHA256 signature of those fields.	Functional	High	Security	AC: `POST /api/qr/pass` returns 201 with a 64-hex-character signature and a PNG data URL; missing fields or a past expiresAt return 400. Test: TC-QR-01, TC-QR-04, TC-QR-05	As built: attendees fetch a freshly signed pass from `GET /api/qr/pass/:ticketId`.
FR-012	The system shall show attendees a pass screen with their name, event details, tier and the QR code.	Functional	High	Attendee	AC: pass screen renders for each ticket of a booking. Test: TC-UX-02, attendee pass route tests	`AttendeePass.jsx`

4.5 Gate check-in

Description: gate staff validate passes and the system records entry once.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-013	The system shall let authorised gate staff scan a pass with the device camera and verify its signature, expiry and event.	Functional	High	Gate Staff	AC: edited payload returns 400 INVALID_SIGNATURE; expired pass returns 400 EXPIRED_QR; pass for another event returns 400 WRONG_EVENT. Test: TC-QR-02, TC-QR-03, TC-SCN-03, TC-SCN-04	Verification uses `crypto.timingSafeEqual`.
FR-014	The system shall record the first valid scan as CHECKED_IN with a timestamp and reject every later scan of the same ticket with DUPLICATE_ENTRY.	Functional	High	Organizer	AC: first scan returns 200 CHECKED_IN; second returns 409 DUPLICATE_ENTRY. Test: TC-SCN-01, TC-SCN-02	As built: `check_in_logs` table with a unique ticket_id; row lock plus `ON CONFLICT DO NOTHING` (DEV-001 resolved).

4.6 Waitlist

Description: queueing for sold-out tiers and reallocation of released seats.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-015	The system shall let an attendee join a FIFO waitlist when a tier has zero available seats.	Functional	Medium	Attendee	AC: join allowed only when available = 0; the same user cannot join the same tier twice. Test: TC-WTL-01	As built: `waitlist_entries` table in PostgreSQL. The server rejects duplicates but does not yet check that the tier is sold out.
FR-016	When seats are released, the system shall notify the next waitlisted user in the app and reserve the seat for 15 minutes.	Functional	Medium	Business	AC: head of queue receives one notification with accept and decline actions; an unclaimed slot moves to the next user after 900 s; a decline reallocates at once. Test: TC-WTL-02, TC-WTL-03	BR-004. As built: in-app notification instead of email (DEV-003).

4.7 Telemetry, reports and administration

Description: live gate figures for organizers and moderation tools for admins.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-017	The system shall show live scan velocity (check-ins per minute) and attendance percentage on the organizer dashboard.	Functional	Medium	Organizer	AC: a new check-in appears on the dashboard within 3 s. Test: TC-ANL-01	SSE
FR-018	The system shall export attendance and revenue summaries as CSV and PDF.	Functional	Medium	Organizer, Admin	AC: CSV row count equals number of tickets; totals match the database. Test: TC-ANL-02	

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-019	The system shall let admins approve, flag or remove events and deactivate accounts.	Functional	High	Admin	AC: a deactivated user's next request returns 401; only ADMIN can call `/api/admin/*`. Test: TC-ADM-01, TC-ADM-02	Actions written to audit_log (SEC-005).

4.8 Business rules

Rule	Definition	Enforced by / Test
BR-001	A user can book at most 5 tickets per transaction for one event.	Booking Engine / TC-BKG-04
BR-002	A hold expires 600 seconds after creation if not confirmed.	Hold expiry worker / TC-BKG-02
BR-003	A pass can be checked in once; later scans are rejected immediately.	Check-in Service / TC-SCN-02
BR-004	A waitlist claim invitation expires after 15 minutes and passes to the next user.	Waitlist Worker / TC-WTL-03
BR-005	An event cannot be submitted for approval without at least one tier with quota > 0.	Event Service / TC-EVT-04
BR-006	Once bookings exist, the event date cannot be changed or the event deleted without sending a cancellation notice to ticket holders.	Event Service / TC-EVT-05

5. Non-functional requirements (detailed)

NFRs are measurable and tied to tests in the STP. IDs NFR-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
NFR-001	95% of catalog search requests shall respond within 150 ms with 500 concurrent users.	Performance	High	500 concurrent users; p95 ≤ 150 ms. Test: TC-PERF-01. Measured p95 16.2 ms with a 1 s pause per user; back-to-back load on one process gives 187.3 ms (DEV-007).
NFR-002	QR verification plus the check-in write shall complete in under 200 ms per scan.	Performance	High	p99 server time < 200 ms over 1,000 sequential scans. Test: TC-PERF-02. Measured server p99 0.84 ms, HTTP p99 12.40 ms.
NFR-003	The booking engine shall handle 200 concurrent hold requests per second with no oversell.	Performance / Integrity	High	200 parallel requests on a 1-seat tier give 1 success and 199 x 409. Test: TC-PERF-03
NFR-004	The system shall be available at least 99.5% of the time during an event's booking window and on the event day.	Reliability	Medium	Health checks `/api/health`, `/live` and `/ready`; availability poller every 60 s with uptime shown to admins. Test: TC-OPS-01.1 to 01.13
NFR-005	An attendee shall be able to go from the home page to a confirmed booking in no more than 4 screens.	Usability	Medium	Task walkthrough with 5 first-time users; all complete in ≤ 4 screens. Test: TC-UX-01. Walkthrough with 5 first-time users: all completed in 4 screens.
NFR-006	Server code shall keep Jest statement, branch and line coverage at or above 80%.	Maintainability	High	`npm test -- --coverage` in CI; build fails below threshold. Current: 92.42% statements, 82.17% branches, 93.67% lines over 36 suites. Test: TC-OPS-02
NFR-007	Booking, check-in and audit records shall be kept for 1 year and survive a server restart.	Data retention	Medium	Reconnect PostgreSQL; all rows present; retention function blocks deletion of records younger than 1 year. Test: TC-OPS-03

5.1 Security

5.1.1 Security objectives

ID	Objective	Threat addressed
SO-1	Only the genuine holder of a confirmed ticket gets through the gate, and only once. A pass that has been copied, edited or replayed is rejected before entry.	Screenshot sharing, forged passes, replay at a second gate.
SO-2	Users can act only within their role. Attendees cannot reach organizer, gate or admin functions, and organizers can change only their own events.	Privilege escalation, insecure direct object references.
SO-3	Passwords, tokens and signing secrets are never exposed in storage, logs, responses or the repository.	Credential theft, secret leakage through git or logs.

5.1.2 Security requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SEC-001	User passwords shall be stored only as bcrypt hashes with cost ≥ 10.	Security	High	DB inspection shows no plaintext; cost parsed from hash ≥ 10. Test: TC-AUTH-03 (SO-3)
SEC-002	RBAC middleware shall block attendee tokens from organizer, gate and admin routes with 403.	Security	High	Every protected route called with each wrong role returns 403. Test: TC-SEC-01 (SO-2)
SEC-003	All input shall be validated and all SQL parameterised to prevent SQL injection and XSS.	Security	High	Injection and script payloads in search, registration and event fields are rejected or stored inert; all SQL parameterised; CSV export escapes formula characters. Test: TC-SEC-02, TC-SEC-04 to 07
SEC-004	Client to server traffic shall use TLS 1.3 outside localhost.	Security	High	`scripts/check-tls.sh`: TLS 1.2 handshake fails, TLS 1.3 succeeds, HTTP redirects to HTTPS, HSTS present (5/5). Test: TC-SEC-03
SEC-005	Failed logins, rejected scans and admin actions shall be written to an audit table with time, user and IP.	Security	Medium	Each listed event creates exactly one audit row; audit rows cannot be updated or deleted. Test: TC-SEC-AUDIT-01 to 12
SEC-006	QR passes shall carry an HMAC-SHA256 signature over a canonical payload, compared in constant time, using a secret loaded only from `QR_HMAC_SECRET`.	Security	High	Any single changed field gives INVALID_SIGNATURE; signing and verification fail closed without the secret. Test: TC-QR-02, TC-QR-SIG-01 to 05 (SO-1, SO-3)

6. Quality attributes and acceptance tests

- **Exit criteria for acceptance:** all High priority FRs implemented and their test cases passed; no open Critical or High defect; NFR-001 to NFR-003 met in the load test; RTM shows every requirement with at least one passed test case.
- **Acceptance test suites:** Authentication, Event Lifecycle, Booking and Concurrency, QR and Gate Check-in, Waitlist, Telemetry and Reports, Administration, Performance, Security, Usability.
- **Acceptance is run by:** the team, with the course faculty observing at the final demo. Automated suites run in GitHub Actions against PostgreSQL 16 and Redis 7 service containers on every push and pull request. Scenario data: one test event "TechSummit 2026" with Early Bird (50), General (300) and VIP (20) tiers.

7. System models and diagrams

7.1 UML use case diagrams

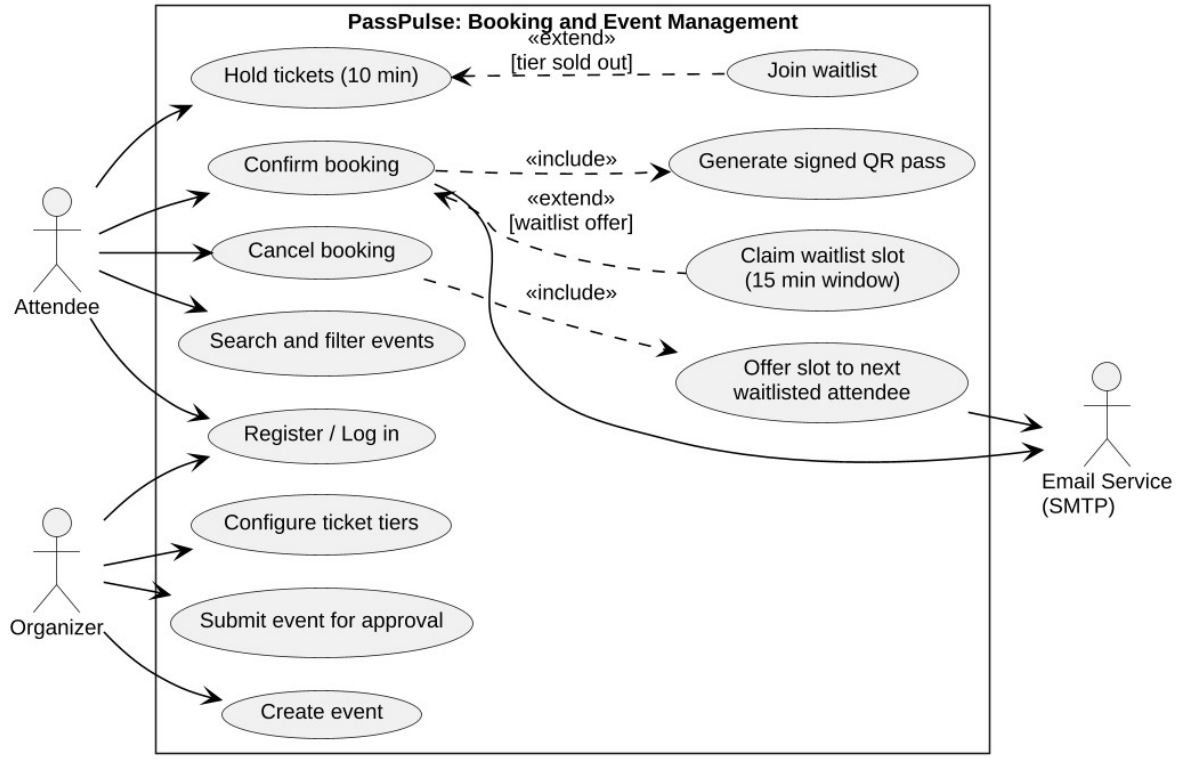


Figure A-1. Use cases for booking and event management (Attendee, Organizer)

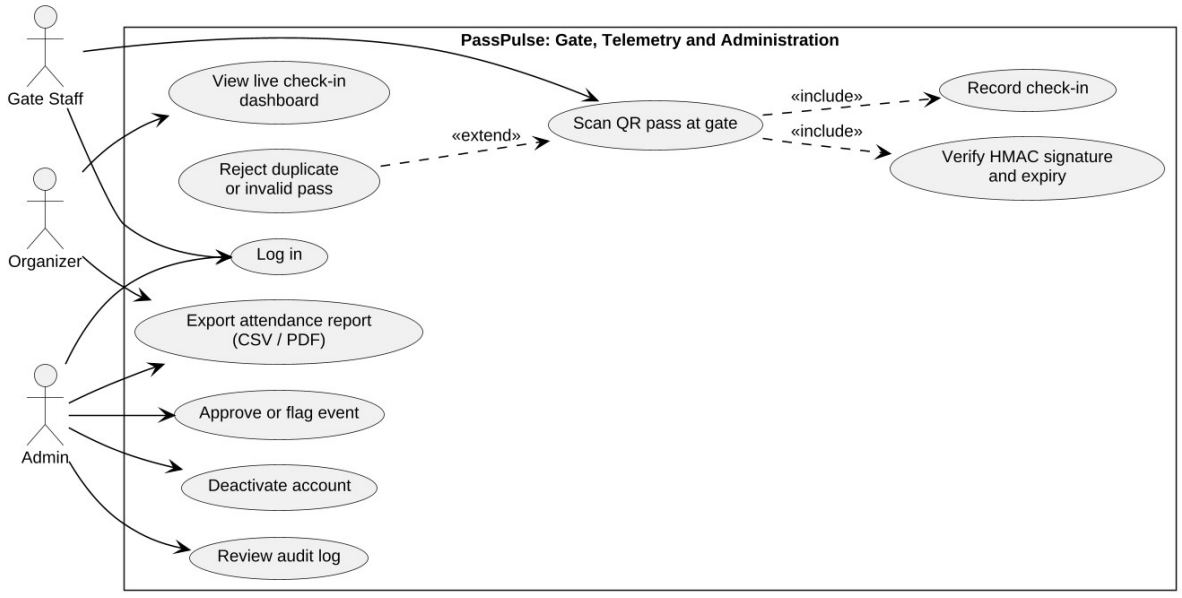


Figure A-2. Use cases for gate check-in, telemetry and administration

7.2 Use case summary

ID	Use case	Actor	Precondition	Main success outcome	Requirements
UC-01	Register / Log in	All	Valid email format	JWT issued	FR-001 to FR-003
UC-02	Search and filter events	Attendee	None	Filtered list shown	FR-004

ID	Use case	Actor	Precondition	Main success outcome	Requirements
UC-03	Create and submit event	Organizer	Logged in as organizer	Event in PENDING_APPROVAL	FR-005 to FR-007
UC-04	Hold tickets	Attendee	Seats available	Hold with 10-minute timer	FR-008, FR-009
UC-05	Confirm booking	Attendee	Active hold	Booking saved, passes issued	FR-010 to FR-012
UC-06	Join / claim waitlist	Attendee	Tier sold out	Queued; later offered seat	FR-015, FR-016
UC-07	Scan pass at gate	Gate Staff	Camera allowed, staff logged in	Attendee marked CHECKED_IN	FR-013, FR-014
UC-08	View dashboard / export	Organizer, Admin	Event PUBLISHED or later	Live figures; CSV/PDF file	FR-017, FR-018
UC-09	Moderate	Admin	Logged in as admin	Event approved or flagged	FR-019

UC-07 detailed flow (Scan pass at gate). Main flow: (1) Staff opens the scanner for the event. (2) Staff points the camera at the attendee's pass. (3) Client decodes the QR text and sends it with the event ID. (4) System checks the signature. (5) System checks expiry. (6) System checks the pass belongs to this event. (7) System checks the ticket has not been used. (8) System records the check-in and shows a green CHECKED_IN screen. Alternate flows: 4a signature mismatch: show red INVALID, log the attempt, return to step 2. 5a pass expired: show red EXPIRED. 6a wrong event: show red WRONG EVENT with the event name on the pass. 7a already used: show red DUPLICATE with the first check-in time. Exception: 3a network down: show "No connection, retry" and do not admit.

8. Requirements Traceability Matrix (RTM)

Status: **N** = not yet implemented, **P** = partly implemented or in review, **A** = implemented and all linked test cases passing. Status at the end of Sprint 2.

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status	Comments
FR-001	Register, unique email	4.1 / SAD 3.4, 4.3	Auth Service	TC-AUTH-01, TC-AUTH-02	A	Registration always ATTENDEE; provisioning by admin
FR-002	bcrypt hashing	4.1 / SAD 3.9	Auth Service	TC-AUTH-03	A	
FR-003	JWT + RBAC	4.1 / SAD 3.9	Auth Middleware	TC-AUTH-04 to 09	A	
FR-004	Catalog filters	4.2 / SAD 4.3	Event Service	TC-EVT-01, TC-EVT-02	A	
FR-005	Create/edit events	4.2 / SAD 3.4	Event Service	TC-EVT-03	A	
FR-006	Lifecycle states	4.2 / SAD Fig. B-3	Event Service	TC-EVT-04	A	
FR-007	Ticket tiers	4.3 / SAD Fig. B-2	Event Service	TC-TCK-01	A	
FR-008	10-min hold	4.3 / SAD Fig. B-4	Booking Engine	TC-BKG-01	A	PG hold plus Redis PXAT key
FR-009	Hold expiry release	4.3 / SAD Fig. B-4	Booking Engine	TC-BKG-02	A	Expiry worker
FR-010	Atomic commit	4.3 / SAD Fig. B-4, ADR-02	Booking Engine	TC-BKG-03, TC-PERF-03	A	
FR-011	Signed QR pass	4.4 / SAD 4.3.1	QR Pass Service	TC-QR-01, TC-QR-04, TC-QR-05	A	Signed on request
FR-012	Pass screen	4.4 / SAD 4.5	Client	TC-UX-02	A	AttendeePass page

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status	Comments
FR-013	Verify at gate	4.5 / SAD Fig. B-5	Check-in Service	TC-QR-02, TC-QR-03, TC-SCN-03, TC-SCN-04	A	Scanner page and RBAC
FR-014	Single check-in	4.5 / SAD Fig. B-5	Check-in Service	TC-SCN-01, TC-SCN-02	A	check_in_logs table
FR-015	Join waitlist	4.6 / SAD 3.6	Waitlist Worker	TC-WTL-01	P	Sold-out check on join not enforced by the server
FR-016	Offer and claim	4.6 / SAD 3.6	Waitlist Worker	TC-WTL-02, TC-WTL-03	A	In-app notification (DEV-003)
FR-017	Live telemetry	4.7 / ADR-04	Telemetry Service	TC-ANL-01	A	SSE
FR-018	CSV/PDF export	4.7	Telemetry Service	TC-ANL-02	A	
FR-019	Admin moderation	4.7 / SAD 3.4	Admin Service	TC-ADM-01, TC-ADM-02	A	
NFR-001	Search p95 ≤ 150 ms	5 / SAD 3.6	Event Service	TC-PERF-01	A	p95 16.2 ms with pause (DEV-007)
NFR-002	Scan < 200 ms	5 / SAD Fig. B-5	Check-in Service	TC-PERF-02	A	Server p99 0.84 ms
NFR-003	No oversell at 200 rps	5 / ADR-02	Booking Engine	TC-PERF-03	A	1 of 200 attendees gets the seat
NFR-004	99.5% availability	5 / SAD 4.4	Platform	TC-OPS-01	A	Health and availability poller
NFR-005	≤ 4 screens to book	5 / SAD 4.5	Client	TC-UX-01	A	5/5 users, 4 screens
NFR-006	Coverage ≥ 80%	5 / STP 14	All server code	TC-OPS-02	A	92.42% / 82.17%
NFR-007	1-year retention	5 / SAD 3.6	PostgreSQL	TC-OPS-03	A	Retention function
SEC-001	bcrypt storage	5.1.2 / SAD 3.9	Auth Service	TC-AUTH-03	A	
SEC-002	RBAC 403	5.1.2 / SAD 3.9	Auth Middleware	TC-SEC-01	A	Applied to /api/qr and /api/checkin
SEC-003	Input validation, SQLi/XSS	5.1.2 / SAD 3.9	All controllers	TC-SEC-02, TC-SEC-04 to 07	A	Includes CSV injection
SEC-004	TLS 1.3	5.1.2 / SAD 3.6	Reverse proxy	TC-SEC-03	A	Nginx, check-tls.sh 5/5
SEC-005	Audit log	5.1.2 / SAD 4.4	All services	TC-SEC-AUDIT-01 to 12	A	Append-only audit_log
SEC-006	HMAC pass, constant-time compare	5.1.2 / SAD 4.3.1	QR Pass Service	TC-QR-02, TC-QR-SIG-05	A	

Software Architecture and Design Specification (SAD)

Revision History

Version	Change Summary
0.1	Component diagram, layering decision, data model
0.2	Booking and check-in sequence diagrams, API definitions, Redis key design
1.0	STRIDE threat model, ADRs, traceability, review comments merged
1.1	Updated to the system as built at the end of Sprint 2. New component, class and sequence diagrams; holds and waitlist moved to PostgreSQL with Redis as a hold projection; SMTP removed; Nginx TLS proxy added; full API listing; open issues closed or carried forward.

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design of PassPulse. It explains how the requirements in SRS v2.2 are split across components, which architectural decisions were taken and why, how the main flows run, and what each API looks like. Version 1.1 describes the system as built at the end of Sprint 2.

1.2 Scope

Covers the React client, the Nginx proxy, the Express API and its services, the PostgreSQL and Redis data stores and the security architecture. Hosting and the GitHub Actions pipeline are described only as far as they affect the design. Payment is a mock step and is not designed here.

1.3 Audience

The development team, team members doing testing, course instructors and evaluators, and reviewers from other teams.

1.4 Definitions

ADR (Architecture Decision Record), API, DTO (Data Transfer Object), HMAC, JWT, RBAC, SSE, STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), TTL. Other terms as in SRS Section 1.4.

2. Document Overview

2.1 How to use this document

Section 3 covers the architecture: goals, component and class diagrams, the chosen pattern, technology, risks, traceability and the threat model. Section 4 covers the design: the event state machine, two sequence diagrams, API definitions, error handling and UX. Readers checking a single requirement should start from the traceability table in Section 3.8.

2.2 Related Documents

- PassPulse SRS v2.2
- PassPulse Software Test Plan v1.1
- PassPulse RTM (SRS Section 8)
- Repository README, `docker-compose.yml` and `docs/DEVIATIONS.md`

3. Architecture

3.1 Goals and Constraints

Goals

- Zero oversold seats under burst load (NFR-003, FR-010).
- Gate decision in under 200 ms, including the database write (NFR-002).
- Passes that cannot be forged or reused (SO-1).
- Each of the four modules can be built and tested by one team member without blocking the others.

Constraints

- Small team, two sprints of development, student hosting budget: one VM or a free-tier container host.
- Stack fixed in the synopsis: React, Node.js/Express, PostgreSQL, Redis, Docker. CI runs on GitHub Actions; Jenkins from the synopsis was not used.
- No real payments; no native mobile apps.

3.2 Stakeholders and Concerns

Stakeholder	Concerns	Addressed in
Attendees	Fair access to seats during a rush; a pass that works at the gate	ADR-02, Fig. B-4, Fig. B-5
Organizers	No overbooking of the hall; live headcount; exports	ADR-02, ADR-04, 4.3
Gate staff	Fast, clear accept/reject; works on a cheap phone	Fig. B-5, 4.5
Admins	Moderation, audit trail	3.4, 3.9, 4.4
Developers	Clear module boundaries, testability, one command to run locally	3.5, 3.6
Evaluators	Traceability from requirement to code and test	3.8, SRS 8

3.3 Component (UML) Diagram

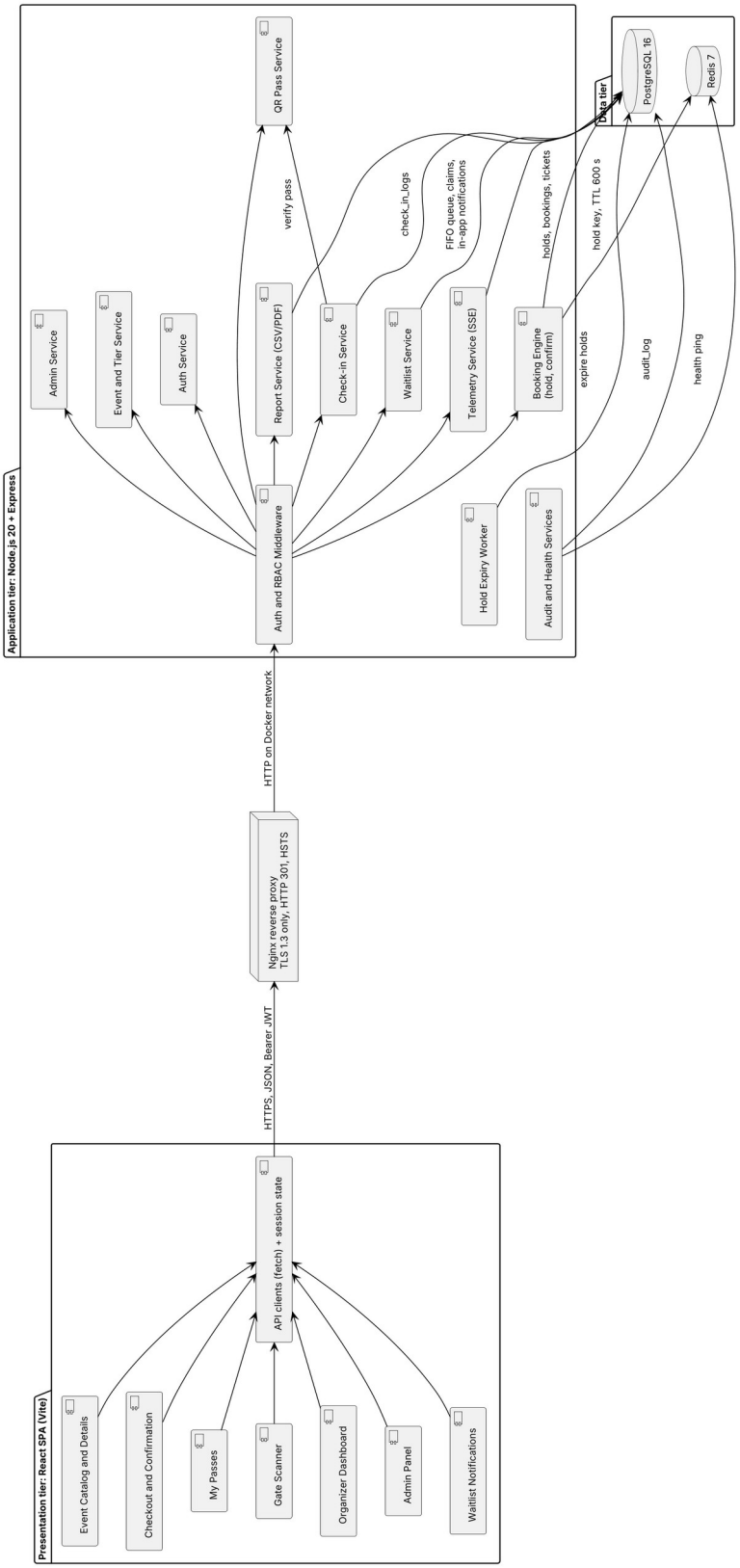


Figure B-1. Component diagram (rotated). Requests pass through the Nginx proxy, then the Auth and RBAC middleware, before reaching a service.

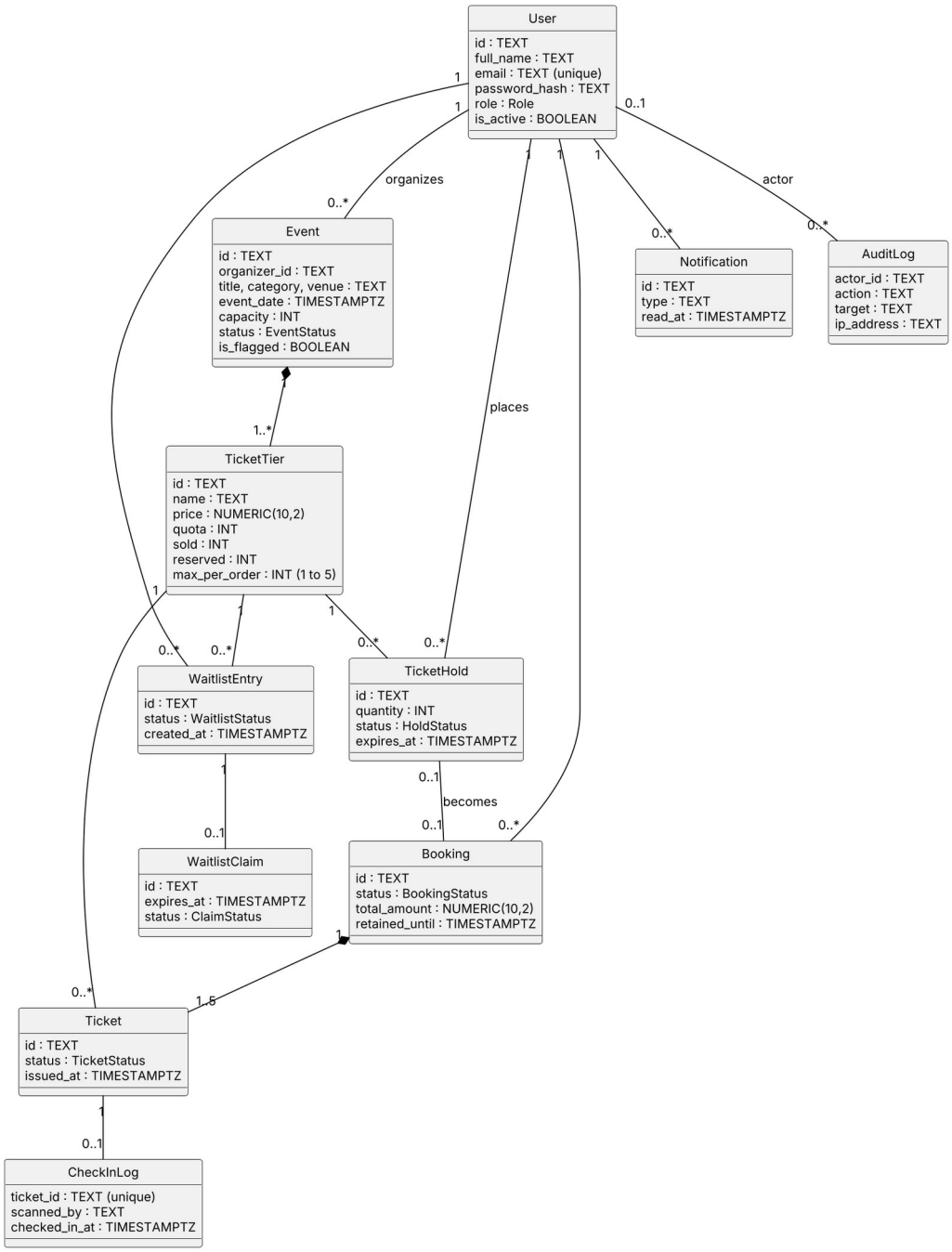


Figure B-2. Class diagram of the domain model (maps to PostgreSQL tables)

3.4 Component Descriptions

Component	Responsibility	Status
React SPA	All screens: catalog (home), event detail, checkout with hold timer, attendee pass, waitlist notifications, organizer dashboard, gate scanner, admin panel. Holds the JWT through AuthContext; API clients add the Authorization header.	Implemented
Nginx proxy	Terminates TLS 1.3, redirects HTTP to HTTPS, sends HSTS, serves the built client and forwards `/api` to the API container.	Implemented
Auth and RBAC Middleware	Verifies the JWT, rejects deactivated accounts, checks the role against a per-route allow list. Rejects with 401 or 403.	Implemented
Auth Service	Registration (always ATTENDEE), login, bcrypt hashing, token issue, admin provisioning of staff accounts.	Implemented
Event Service	Event CRUD, tier setup, lifecycle transitions, cached catalog search. Data in an in-memory repository (DEV-006).	Implemented

Component	Responsibility	Status
Booking Engine	Creates holds in PostgreSQL and mirrors them to Redis; confirms bookings in one PostgreSQL transaction; issues tickets.	Implemented
Hold Expiry Worker	Runs every second inside the API process, releases expired holds with `FOR UPDATE SKIP LOCKED`, hands released seats to the waitlist and removes stale Redis keys.	Implemented
Waitlist Service	FIFO queue per tier, 15-minute claims, accept and decline, in-app notifications (DEV-003).	Implemented
QR Pass Service	Builds the canonical payload, signs it with HMAC-SHA256, renders the QR image and verifies scanned passes. Passes are signed when the attendee requests them.	Implemented
Check-in Service	Verifies the pass, reads ticket and booking status from the database, records the first scan in check_in_logs and audits every rejection.	Implemented
Telemetry Service	Aggregates check-ins per minute and pushes updates over SSE.	Implemented
Report Service	Attendance and revenue summaries; CSV export with formula escaping; PDF export.	Implemented
Admin Service	Approve, reject, flag and remove events; deactivate and reactivate users; audit log and availability views.	Implemented
Audit and Health	Append-only audit_log writer; `/api/health`, `/live`, `/ready` and a 60 s availability poller.	Implemented
PostgreSQL 16	System of record for tiers mirror, bookings, tickets, holds, waitlist, claims, notifications, check-ins and audit log.	In use
Redis 7	Projection of active holds with absolute expiry (PXAT).	In use

3.5 Chosen Architecture Pattern and Rationale

Layered client-server architecture, deployed as one API process (modular monolith). The client is the presentation layer. On the server each module has three layers: routes and controllers (HTTP, validation, status codes), services (business rules), and data access (SQL and Redis calls). A layer only calls the one below it. This is already the structure in the repository: `routes/`, `controllers/`, `services/`, `models/`.

Reasons: each team member owns one vertical slice and can unit test the service layer without HTTP (for example `qr.service.test.js` and `checkin.service.test.js`). One process keeps transactions simple, which matters for the booking commit. Deployment is one `docker-compose up`.

Rejected alternatives. Microservices: four people cannot run and monitor nine services, and a booking would need a distributed transaction or saga between booking, inventory and pass services. Serverless functions: cold starts would break the 200 ms gate target, and the hold expiry worker and SSE streams need a long-running process.

Architecture Decision Records

ADR	Decision	Context and reason	Consequence
ADR-01	Layered modular monolith	See above.	Must keep module boundaries by code review, since nothing enforces them at runtime.
ADR-02	Durable hold in PostgreSQL with a Redis projection (revised in v1.1)	A row lock held for 10 minutes would block the tier, and a Redis-only hold is lost on restart. Each hold is a `ticket_holds` row that increments `ticket_tiers.reserved` under a row lock, with `CHECK (sold + reserved <= quota)`. Redis gets `SET hold:{id} NX PXAT` for fast lookups.	A worker releases expired holds and reconciles Redis every second. If the Redis write fails, the PostgreSQL hold is rolled back.

ADR	Decision	Context and reason	Consequence
ADR-03	HMAC-SHA256 signed JSON payload in the QR, not a JWT and not a bare ticket ID	A bare ID can be guessed and copied. A JWT works but is longer, making a denser QR that scans slower on low-end phones. HMAC over five fields gives a short payload and tamper detection with one secret.	Secret rotation invalidates all issued passes; rotation only between events.
ADR-04	Server-Sent Events for live dashboard	Updates go one way, server to organizer. SSE runs over plain HTTP and reconnects automatically; WebSockets add a second protocol for no gain here.	Max about six SSE connections per browser per domain over HTTP/1.1; fine for one dashboard tab.

3.6 Technology Stack and Data Stores

Layer	Technology (version from package.json / compose)
Client	React 19, Vite, Tailwind CSS, html5-qrcode; linted with oxlint
API	Node.js 20, Express, jsonwebtoken, bcryptjs, qrcode, pg, redis, dotenv, cors
Proxy	Nginx with TLS 1.3 only, HSTS
Data	PostgreSQL 16-alpine, Redis 7-alpine, both with named Docker volumes
Testing	Jest, Supertest, pg-mem for unit-level SQL, real PostgreSQL and Redis for integration suites, a Node load benchmark and k6 script
CI / quality	GitHub Actions with PostgreSQL 16 and Redis 7 service containers; 80% coverage gate; oxlint and vite build for the client

Redis key design

Key	Type	Expiry	Purpose
`hold:{holdId}`	String (JSON)	PXAT = hold expiresAt	Fast projection of an active hold (FR-008). Written with NX so a hold ID is never reused.

The per-tier counter and waitlist lists planned in v1.0 were dropped. Availability and the waitlist live in PostgreSQL so they survive a Redis restart.

PostgreSQL tables: users, events, ticket_tiers (with a `reserved` column), bookings, tickets (one row per QR pass), ticket_holds, waitlist_entries, waitlist_claims, notifications, check_in_logs (unique index on ticket_id, which enforces BR-003 at the database level) and audit_log (append-only, protected by triggers). Schema in `server/db/schema.sql` plus two migrations. Users and events are mirrored into their tables when a hold needs them (DEV-002, DEV-006). Foreign keys as in Figure B-2.

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Redis restarts and loses hold keys mid-checkout	Low	Holds are durable in PostgreSQL; the worker rebuilds or removes Redis keys. Worst case is a slower lookup, never an oversell.
Gate network is slow or drops at the venue	High	Small request body (QR text only), 3 s client timeout with a retry button; never admit on timeout.
`QR_HMAC_SECRET` leaks	High	Secret only in env and the CI secret store; `.env` in .gitignore; rotate between events.
Users and events are in memory (DEV-002, DEV-006)	Medium	Restart reloads seed data; bookings, tickets and check-ins are already durable and reference mirrored rows. Schema is ready for the switch.
CORS allows all origins	Medium	The proxy serves client and API from one origin; restricting the CORS list is open item OI-4.
One member falls behind and blocks integration	Medium	Services exchange plain DTOs; each module has stubs so others can integrate early.

3.8 Traceability to Requirements

Requirement	Component	Design reference
FR-001 to FR-003, SEC-001, SEC-002	Auth Service, Auth Middleware	3.4, 3.9, 4.3.4
FR-004 to FR-007, BR-005, BR-006	Event Service	3.4, Fig. B-2, Fig. B-3
FR-008 to FR-010, NFR-003, BR-001, BR-002	Booking Engine, Hold Expiry Worker, Redis, PostgreSQL	ADR-02, Fig. B-4, 4.3.3
FR-011, FR-012, SEC-006	QR Pass Service, Client	ADR-03, 4.3.1, Fig. B-4
FR-013, FR-014, NFR-002, BR-003	Check-in Service	Fig. B-5, 4.3.2
FR-015, FR-016, BR-004	Waitlist Service, Hold Expiry Worker	3.6 (tables), 4.3.3
FR-017, FR-018	Telemetry Service, Report Service	ADR-04, 4.3.5
FR-019, SEC-005	Admin Service, audit_log	3.4, 4.3.5, 4.4
NFR-004, SEC-004	Audit and Health, Nginx proxy	3.4, 4.4

3.9 Security Architecture

Trust boundaries. (1) Between the browser and the proxy: TLS 1.3 only. Nothing from the client is trusted, including role claims in the body; self-registration cannot pick a role. (2) Between the API and the data stores: only the API container can reach PostgreSQL and Redis on the Docker network; their ports are not published in production. (3) Between gate devices and the API: a gate device is a client like any other and must hold a GATE_STAFF, STAFF or ADMIN token.

Threat model (STRIDE)

Threat	Example against PassPulse	Control	Req
Spoofing	Attendee forges a token claiming ORGANIZER	JWT signed with `JWT_SECRET`; role read from DB on each request, not from the body	FR-003, SEC-002
Tampering	Attendee edits the eventId or expiresAt in a pass	HMAC-SHA256 over canonical payload; constant-time compare	SEC-006
Repudiation	Gate staff claims a guest was never admitted	check_in_logs stores ticket and time; audit_log rows for rejected scans, failed logins and admin actions, append-only	SEC-005
Information disclosure	Password hashes or secrets in logs or API responses	bcrypt; response DTOs exclude password_hash; secrets only in env	SEC-001, SO-3
Denial of service	Bot floods the hold endpoint to lock all seats	Max 5 seats per order (BR-001); holds expire after 600 s and are released by the worker	NFR-003, BR-001
Elevation of privilege	Organizer edits another organizer's event by changing the ID in the URL	Service checks `event.organizer_id = req.user.id` before any write	FR-005, SEC-002

Replay of a valid pass is handled by the single-use check (BR-003) rather than by the signature, since a screenshot of a genuine pass carries a genuine signature.

4. Design

4.1 Design Overview

Each module follows the same pattern: a route file declares the path, a controller validates the body and maps results to HTTP codes, and a service holds the logic and returns plain result objects such as

`{ success, reason }`. Controllers never throw business errors to Express; they translate reason codes using a lookup table. Event state changes are restricted to the transitions in Figure B-3.

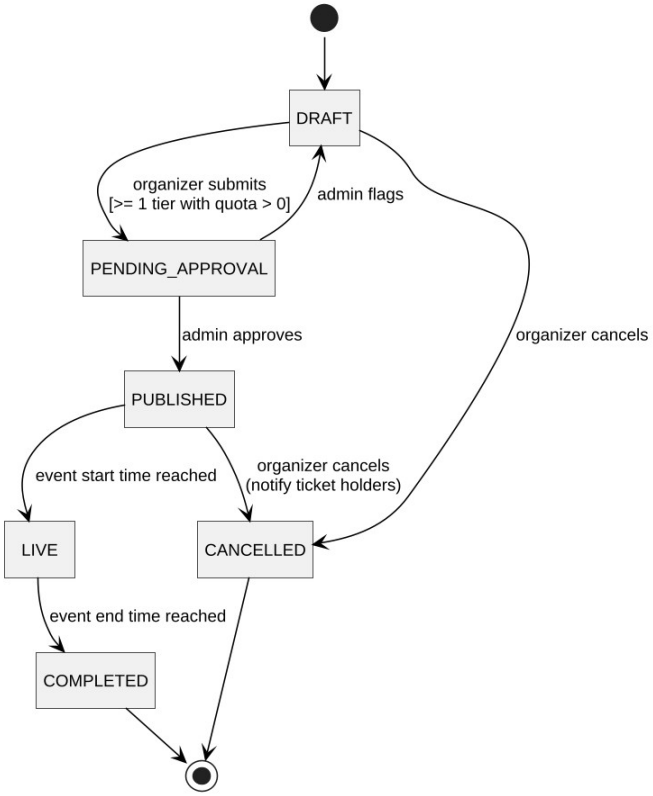


Figure B-3. Event lifecycle state machine (FR-006, BR-005)

4.2 UML Sequence Diagrams

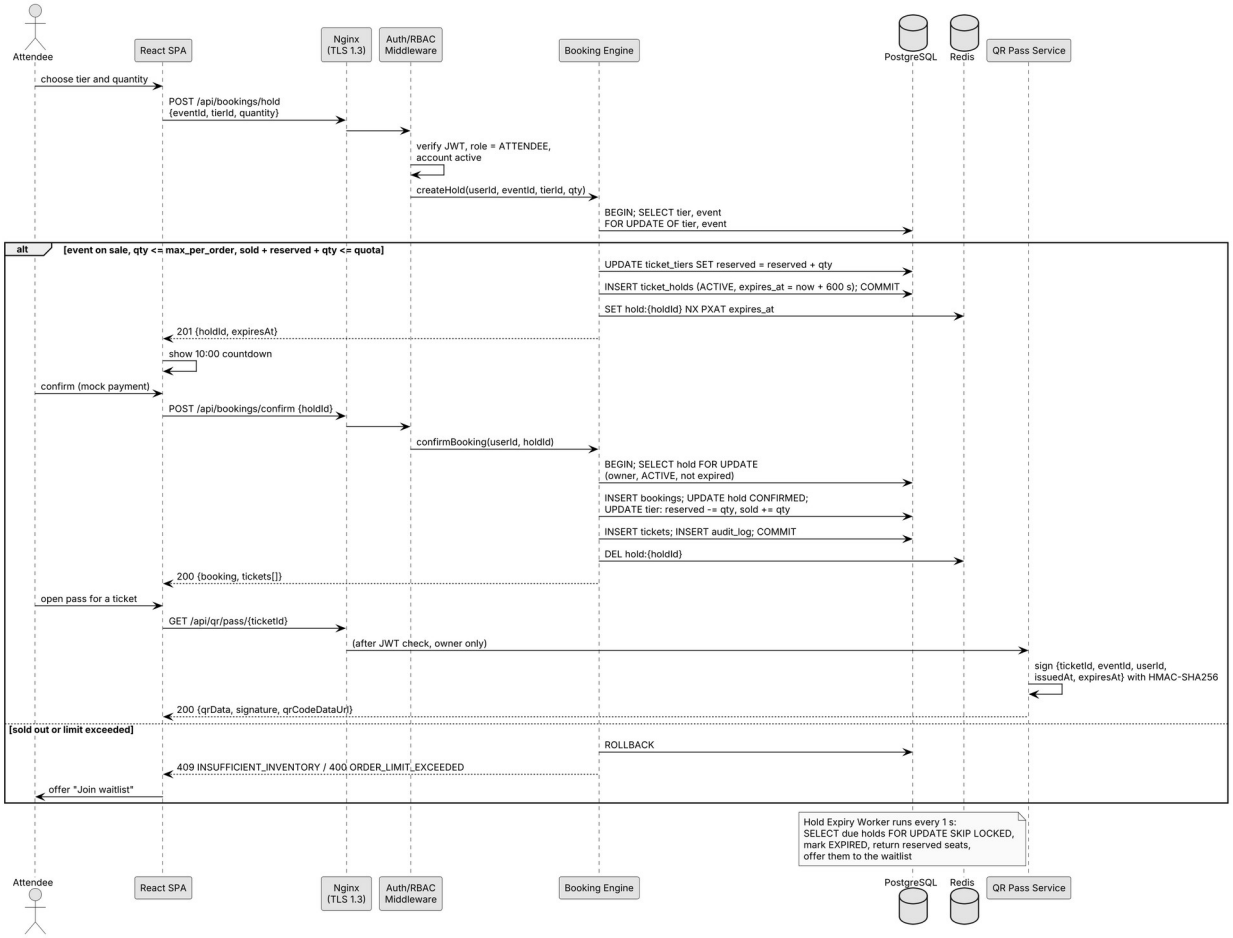


Figure B-4. Hold, confirm and pass request (FR-008 to FR-012)

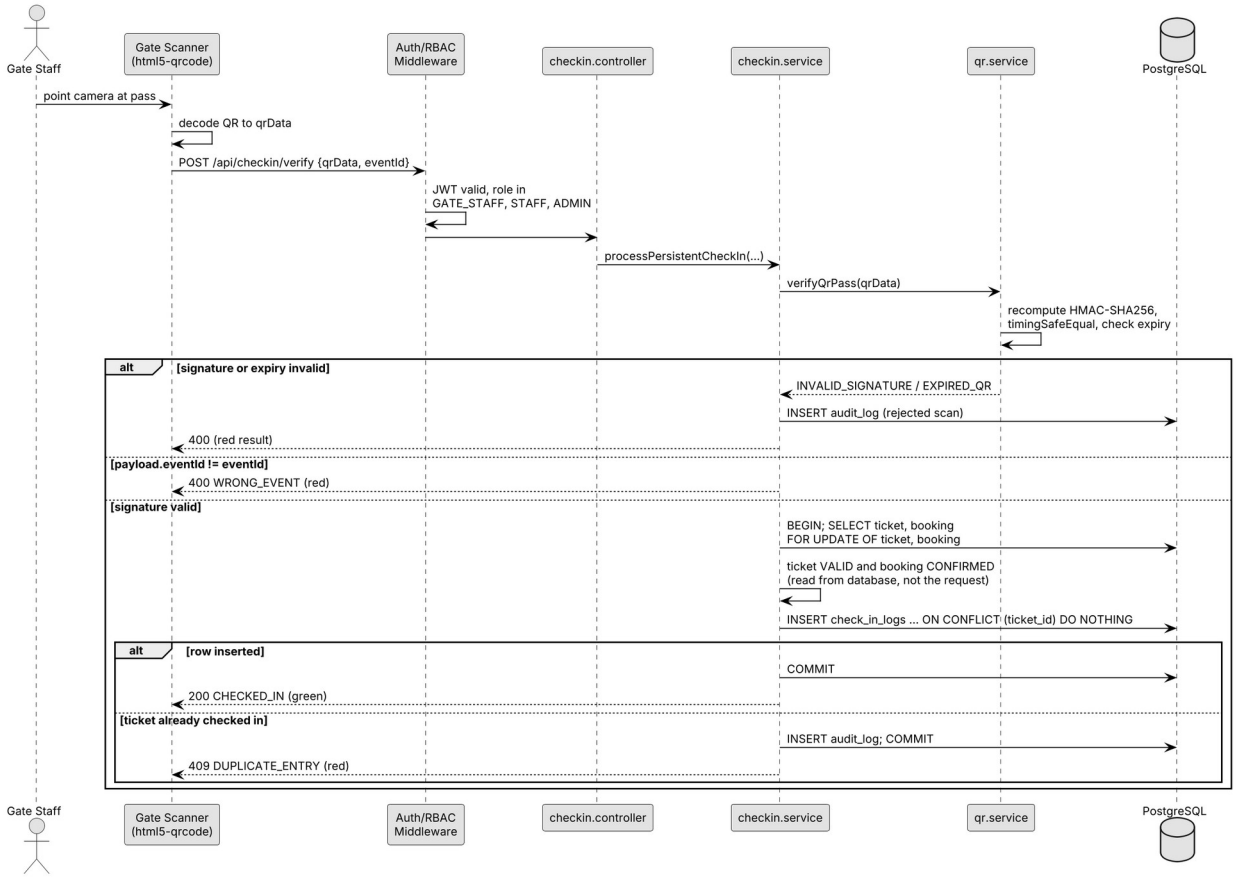


Figure B-5. Gate check-in with RBAC and persistent single use (FR-013, FR-014)

4.3 API Design

Base path `/api`. All bodies are JSON. Errors use `{ "status": "error", "code": "...", "message": "..." }`. All endpoints below exist on `main`.

4.3.1 QR Pass Service

Endpoint	Request / Response	Access / Errors
`GET /api/qr/pass/:ticketId`	200 `{ ticketId, eventId, userId, issuedAt, expiresAt, signature, qrData, qrCodeDataUrl }`, signed at request time	Authenticated; owner only (403 FORBIDDEN otherwise). 404 TICKET_NOT_FOUND
`GET /api/qr/tickets`	200 list of the caller's tickets	Authenticated
`POST /api/qr/pass`	Req `{ ticketId }`; 201 signed pass built only from the stored ticket record	STAFF, ADMIN. 400 INVALID_QR_REQUEST, UNCONFIRMED_BOOKING, INVALID_TICKET_STATUS, EXPIRED_PASS; 404 TICKET_NOT_FOUND

Signature: `HMAC_SHA256(QR_HMAC_SECRET, JSON.stringify({ticketId, eventId, userId, issuedAt, expiresAt}))` as 64 hex characters. Key order is fixed by `serializePayload()`. Signing and verification fail closed if the secret is not set.

4.3.2 Check-in Service

Endpoint	`POST /api/checkin/verify`
Access	GATE_STAFF, STAFF, ADMIN (401 without a token, 403 for other roles)
Request	`{ qrData, eventId, ticketStatus, bookingConfirmationStatus }`

Behaviour	When PostgreSQL is available, ticket and booking status are read from the database and the two status fields in the body are ignored (OI-2 resolved). The ticket and booking rows are locked, and the check-in is inserted with `ON CONFLICT (ticket_id) DO NOTHING`.
Response 200	`{ status: "success", code: "CHECKED_IN", data: { ticketId, eventId, userId, checkedInAt } }`
Errors	400 INVALID_CHECKIN_REQUEST, INVALID_QR_FORMAT, MISSING_QR_FIELDS, INVALID_SIGNATURE, EXPIRED_QR, WRONG_EVENT, INVALID_TICKET_STATUS, UNCONFIRMED_BOOKING; 404 TICKET_NOT_FOUND; 409 DUPLICATE_ENTRY. Every rejection writes an audit row.

4.3.3 Booking Engine and Waitlist

Endpoint	Request / Response	Errors
`POST /api/bookings/hold`	Req `{ eventId, tierId, quantity }` 201 `{ hold: { holdId, expiresAt, ... } }`	400 INVALID_HOLD_REQUEST, ORDER_LIMIT_EXCEEDED; 404 EVENT_NOT_FOUND, TIER_NOT_FOUND; 409 INSUFFICIENT_INVENTORY, EVENT_NOT_ON_SALE
`POST /api/bookings/confirm`	Req `{ holdId }` 200 `{ booking: { bookingId, tickets, total } }`; repeat calls return the same booking	400 INVALID_CONFIRMATION_REQUEST; 403 HOLD_NOT_OWNED; 404 HOLD_NOT_FOUND; 409 HOLD_NOT_ACTIVE (expired or released), EVENT_NOT_ON_SALE, INVENTORY_UNAVAILABLE
`POST /api/bookings/waitlist`	Req `{ eventId, tierId, quantity }`; 201 waitlist entry	400 ORDER_LIMIT_EXCEEDED; 409 ACTIVE_WAITLIST_ENTRY_EXISTS, EVENT_NOT_AVAILABLE
`GET /api/bookings/notifications`	200 list of the caller's notifications and open claims	
`POST /api/bookings/claims/:claimId/accept`	200 claim with a hold to confirm	403 CLAIM_NOT_OWNED; 404 CLAIM_NOT_FOUND; 409 CLAIM_EXPIRED, CLAIM_NOT_ACTIVE
`POST /api/bookings/claims/:claimId/decline`	200; seat offered to the next user at once	409 CLAIM_NOT_ACTIVE

All booking routes require the ATTENDEE role.

4.3.4 Auth Service

Endpoint	Request / Response	Access / Errors
`POST /api/auth/register`	Req `{ fullName, email, password }` 201 user with role ATTENDEE	Public. 400 on missing or invalid fields; 409 if the email exists. Any role in the body is ignored.
`POST /api/auth/login`	Req `{ email, password }` 200 `{ token, user }`	Public. 400 on missing fields; 401 on bad credentials or ACCOUNT_DEACTIVATED
`GET /api/auth/me`	200 current user	Authenticated

Endpoint	Request / Response	Access / Errors
`POST /api/auth/provision`	Req `{ fullName, email, password, role }` 201 staff account	ADMIN. 400 on invalid role or fields; 409 if the email exists

4.3.5 Events, administration, telemetry, reports and health

Area	Endpoints	Access
Events	`GET /api/events` (filters), `/categories`, `/:id`	Public
Events	`GET /mine`, `POST /`, `GET /:id/manage`, `PUT /:id`, `POST /:id/cancel`, `PATCH /:id/status`, tier create, update and delete	ORGANIZER (own events), ADMIN
Admin	Events list, approve, reject, flag, unflag, remove; users list, deactivate, reactivate; `/audit-logs`; `/availability`	ADMIN
Telemetry	`GET /api/telemetry/:eventId`, `/:eventId/stream` (SSE, token may be passed as a query parameter)	ORGANIZER, ADMIN
Reports	`GET /api/reports/:eventId/attendance`, `/revenue`, `/export` (CSV or PDF)	ORGANIZER, ADMIN
Health	`GET /api/health`, `/live`, `/ready`	Public

4.4 Error Handling, Logging and Monitoring

- **Error codes:** upper-case strings defined next to the service that raises them; controllers map codes to HTTP status with a lookup object (for example `statusByReason` in the check-in controller) or a status carried on the error. Unknown errors reach the Express error middleware, which returns 500 with a generic message and no stack trace.
- **Fail closed:** any verification error at the gate rejects entry. The scanner never shows green unless the API returns 200.
- **Logging:** JSON lines with timestamp, level, requestId, route, userId and code. Never logged: passwords, JWTs, `QR_HMAC_SECRET`, full qrData. The error handler currently logs with `console.error`.
- **Audit:** failed logins, rejected scans (with reason), confirmed bookings, provisioning and admin actions go to the audit_log table (SEC-005). Triggers block UPDATE and DELETE on audit rows; records are kept for at least 1 year (NFR-007).
- **Monitoring:** `/api/health`, `/live` and `/ready` report database and Redis status; an availability poller samples every 60 s and admins see uptime at `/api/admin/availability`. Search responses carry a Server-Timing header.

4.5 UX Design

- Booking path is Catalog (home), Event detail, Checkout with timer, Pass: four screens (NFR-005).
- The hold timer is always visible on checkout and turns bold under 60 seconds. On expiry the page explains the seats were released and offers to try again.
- Scanner result fills the screen with a large word (CHECKED IN, DUPLICATE, EXPIRED, WRONG EVENT, INVALID) and a colour, so staff can read it at arm's length and colour-blind staff are not dependent on colour. A short beep distinguishes accept from reject.
- The pass page shows the QR at a fixed minimum size and is fetched freshly signed each time it opens.
- Forms show errors next to the field, in text, after the user leaves the field.

4.6 Open Issues and Next Steps

ID	Issue	Status
OI-1	Check-in records were kept in a JavaScript Map and lost on restart.	Resolved: check_in_logs with unique ticket_id (DEV-001)
OI-2	The verify endpoint trusted ticketStatus and bookingConfirmationStatus from the request body.	Resolved: read from the database
OI-3	`/api/qr` and `/api/checkin` were not behind the Auth/RBAC middleware.	Resolved: role checks on both
OI-4	CORS allows all origins. Restrict to the client origin in production.	Open
OI-5	Decide whether passes should rotate every 30 s to stop screenshot sharing before the gate; current design relies on single use.	Open, after demo
OI-6	Move users and events from the in-memory repository to their PostgreSQL tables (DEV-002, DEV-006).	Open
OI-7	The check-in record does not yet store the scanning staff member's ID.	Open

4.7 Deviations from SAD v1.0

ID	Deviation	Status
DEV-001	Check-in records in memory	Resolved
DEV-002	Users served from an in-memory repository	Approved
DEV-003	Waitlist offers as in-app notifications instead of SMTP email	Approved
DEV-004	Client checked by oxlint and vite build; no React unit test runner	Approved
DEV-005	No root-level workspace script; client and server run separately	Approved
DEV-006	Events and tiers served from an in-memory repository	Approved (temporary)
DEV-007	NFR-001 measured with a 1 s pause per user	Approved
DEV-008	Booking journey needed the booking API	Resolved; walkthrough tracked in issue #36

Full entries in `docs/DEVIATIONS.md`.

5. Appendices

5.1 Glossary: ADR, API, DTO, HMAC, JWT, RBAC, SSE, STRIDE, TTL (see 1.4 and SRS 1.4).

5.2 References: ISO/IEC/IEEE 42010:2022 Architecture description; OWASP Top 10 (2021); OWASP ASVS 4.0; NIST SP 800-160 Vol. 1; RFC 2104 (HMAC); RFC 7519 (JWT); Sommerville, Software Engineering, 10th ed., Ch. 6.

5.3 Tools: PlantUML (all diagrams in this document), draw.io for early sketches, Supertest suites in the repository exercise every API in 4.3.

Software Test Plan (STP)

Revision History

Version	Change Summary
0.1	Validation specification (initial planning)
1.0	Full test plan on course template; test case catalogue; first execution results for QR and check-in modules
1.1	Updated after Sprint 2. Test items now merged; tools changed to what was actually used (GitHub Actions, pg-mem, real PostgreSQL and Redis, Node benchmark); catalogue statuses and evidence updated; test cases added during the sprints listed with a mapping for renumbered security IDs; final metrics.

1. Introduction

Purpose: This plan defines how PassPulse v2.2 is tested: objectives, scope, levels, test cases, environment, schedule, roles and the criteria for starting, stopping and accepting testing.

Scope: All functional requirements FR-001 to FR-019, NFR-001 to NFR-007, SEC-001 to SEC-006 and business rules BR-001 to BR-006 from SRS v2.2. Excluded items are listed in Section 4.

References: PassPulse SRS v2.2; PassPulse SAD v1.1; IEEE 829-2008 Standard for Software Test Documentation; OWASP Web Security Testing Guide v4.2; `docs/TEST_CASES_UPDATED.md`, `docs/UNIT_TEST_REPORT.md` and `docs/SYSTEM_VALIDATION_REPORT.md` in the repository.

Definitions: SRS, SAD, RTM, SUT (System Under Test), UAT (User Acceptance Test), TC (Test Case), p95/p99 (95th/99th percentile latency), VU (virtual user).

2. Test Items

Item	Source location	State
Auth and RBAC module	server/src/controllers/auth.controller.js, middleware, routes/auth.routes.js	Merged
Event and tier module	server/src/services (event, tier, lifecycle), routes/event.routes.js	Merged
Booking engine and hold expiry worker	server/src/services/booking-hold.service.js, booking-confirmation.service.js, hold-expiry.worker.js	Merged
Waitlist	server/src/services/waitlist.service.js, migration 0002	Merged
QR pass service and API	server/src/services/qr.service.js, routes/qr.routes.js	Merged
Check-in service and API	server/src/services/checkin.service.js, routes/checkin.routes.js	Merged
Telemetry, reports, admin, audit, health	server/src/services (telemetry, report, admin, audit, health, availability)	Merged
Database schema	server/db/schema.sql, server/db/migrations	Merged
React client	client/src	Merged
Deployment configuration	docker-compose.yml, Dockerfiles, nginx config, scripts/check-tls.sh	Merged

3. Features to be Tested

Req ID	Feature	Test cases
FR-001 to FR-003	Registration, password hashing, JWT and RBAC	TC-AUTH-01 to TC-AUTH-09, TC-AUTH-SEC-01 to 03, TC-MID-01 to 05, TC-PROV-01 to 05, TC-SEC-01
FR-004 to FR-006	Catalog filters, event CRUD, lifecycle transitions	TC-EVT-01 to TC-EVT-05
FR-007	Ticket tier configuration	TC-TCK-01

Req ID	Feature	Test cases
FR-008 to FR-010	10-minute hold, expiry release, atomic commit	TC-BKG-01 to TC-BKG-04
FR-011, FR-012	Signed QR pass generation and pass screen	TC-QR-01 to TC-QR-06, TC-QR-SIG-01 to 05, TC-UX-02
FR-013, FR-014	Gate verification and single check-in	TC-SCN-01 to TC-SCN-05
FR-015, FR-016	Waitlist join, offer and claim window	TC-WTL-01 to TC-WTL-03
FR-017 to FR-019	Live telemetry, exports, admin moderation	TC-ANL-01, TC-ANL-02, TC-ADM-01, TC-ADM-02
NFR-001 to NFR-003	Search latency, scan latency, booking concurrency	TC-PERF-01 to TC-PERF-03, TC-PERF-02.1 to 02.4
NFR-004, NFR-006, NFR-007	Availability, coverage, data retention	TC-OPS-01.1 to 01.13, TC-OPS-02, TC-OPS-03
NFR-005	Screens to complete a booking	TC-UX-01
SEC-001 to SEC-006	Security requirements	TC-SEC-01 to TC-SEC-07, TC-SEC-AUDIT-01 to 12, TC-QR-SIG-05, TC-SEED-01, TC-SEED-02, TC-AUTH-03, TC-QR-02

4. Features Not to be Tested

- Payment processing: checkout uses a mock confirmation; no gateway exists to test.
- Internal behaviour of PostgreSQL, Redis and the qrcode library; they are used as black boxes and only their integration is tested.
- Email delivery: the build sends waitlist offers as in-app notifications (DEV-003), so there is no email to test.
- React component unit tests: the client is checked by `oxlint`, `vite build` and manual walkthroughs (DEV-004).
- Browser compatibility beyond the four browsers in SRS 2.4; native app stores.
- Camera hardware quality. Scanner tests feed known QR images through the decoder.

5. Test Approach / Strategy

Levels

- **Unit:** Jest tests on service functions with no HTTP and no network (for example `verifyQrPass`, `processCheckIn`), using pg-mem where SQL is involved. Written by the module owner in the same pull request as the code.
- **Integration:** Supertest against the Express app, and `*.postgres.test.js` suites against real PostgreSQL 16 and Redis 7 service containers in GitHub Actions.
- **System:** end-to-end flow on the Docker Compose stack: log in, hold, confirm, fetch pass, scan at the gate, duplicate scan rejected.
- **Acceptance:** scenario walkthrough by the whole team with the faculty demo script, using the TechSummit 2026 data set.

Types

- Functional (black box): equivalence classes and boundary values, e.g. qty 0, 1, 5, 6 for BR-001; holds checked just before and after their deadline.
- Negative: missing fields, wrong role, tampered pass, expired pass, wrong event.
- Regression: the full Jest suite runs on every push and pull request; merge blocked on failure.

- Concurrency and performance: Jest performance suites and a Node benchmark for NFR-001 and NFR-002 (a k6 script is kept in `docs/perf`); a 200-attendee race test for NFR-003.
- Usability: task-based walkthrough with five first-time users for NFR-005.
- Static analysis: `oxlint` on the client in CI; coverage gate of 80% on the server.

Entry criteria: feature branch opened as a pull request to `main` with its tests passing in CI; containers start from docker-compose; test data seed script runs without error.

Exit criteria: 100% of planned test cases executed; all High priority test cases passed; 0 open Critical defects and no more than 2 open Major defects with a documented workaround; statement and branch coverage ≥ 80%; NFR-001 to NFR-003 targets met in the final performance run.

5.1 Security Validation

Req	What is validated	Method / tool	Result
SEC-006	Edited pass fields rejected; signature compared in constant time; service fails closed without QR_HMAC_SECRET	Jest (TC-QR-02, TC-QR-SIG-01 to 05)	Pass
BR-003	Replay of a genuine pass rejected on second scan, including two concurrent scans, enforced by the unique index on check_in_logs	Supertest, PostgreSQL suite	Pass
SEC-002	Protected routes called with wrong roles return 403; organizer cannot edit another organizer's event; self-registration cannot pick a role	Supertest role matrix with tokens for each role (TC-SEC-01, TC-AUTH-SEC-01 to 03)	Pass
SEC-003	SQL injection and XSS payloads in search, event fields and registration; CSV formula injection in exports	Supertest payload lists (TC-SEC-04 to 07), csv.util tests	Pass
SEC-001	No plaintext passwords; bcrypt cost ≥ 10; seeded accounts hashed	Jest (TC-AUTH-03, TC-SEED-01, 02)	Pass
SEC-004	TLS 1.3 only; HTTP redirects to HTTPS; HSTS	`scripts/check-tls.sh` against the Nginx container	Pass (5/5)
SEC-005	Failed logins, rejected scans and admin actions written to audit_log; audit rows cannot be changed	Supertest and schema tests (TC-SEC-AUDIT-01 to 12)	Pass

OWASP ZAP and gitleaks from version 1.0 were not run; the payload suites above replaced the ZAP baseline.

5.2 Test Case Catalogue

Priority H/M/L. Auto = automated in Jest/Supertest or k6; Man = manual. Status at the end of Sprint 2: Pass, Fail, or NR (not run). Evidence is the suite named in Section 13.

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-AUTH-01	FR-001	Register with valid data: 201, user row created with role ATTENDEE	Integ	H	Auto	Pass
TC-AUTH-02	FR-001	Register with an existing email: 409	Integ	H	Auto	Pass
TC-AUTH-03	FR-002, SEC-001	Stored password is a bcrypt hash with cost ≥ 10; plaintext not found in DB dump or logs	Integ	H	Auto	Pass
TC-AUTH-04	FR-003	Protected route with no token or a token signed with another secret: 401	Integ	H	Auto	Pass
TC-AUTH-05	FR-003	Attendee token on POST /api/events: 403	Integ	H	Auto	Pass
TC-EVT-01	FR-004	Filter by category "Tech": only Tech events returned	Integ	H	Auto	Pass
TC-EVT-02	FR-004	Combined filters (date range + max price + venue); DRAFT events never returned	Integ	H	Auto	Pass
TC-EVT-03	FR-005	Organizer A cannot edit organizer B's event: 403	Integ	H	Auto	Pass

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-EVT-04	FR-006, BR-005	DRAFT to LIVE rejected 409; submit with no tier rejected	Unit	M	Auto	Pass
TC-EVT-05	BR-006	Change date of an event with bookings: notice queued to all holders	Integ	M	Auto	Pass
TC-TCK-01	FR-007	Tier with quota 0 or price -1: 400	Unit	H	Auto	Pass
TC-BKG-01	FR-008	Hold 2 seats: 201, expiresAt = now + 600 s, availability reduced by 2	Integ	H	Auto	Pass
TC-BKG-02	FR-009, BR-002	Let a hold pass its deadline: expiry worker releases it once and availability is restored	Integ	H	Auto	Pass
TC-BKG-03	FR-010	Confirm after quota changed underneath: transaction rolls back, 409, no booking row	Integ	H	Auto	Pass
TC-BKG-04	BR-001	Boundary qty 5 accepted, qty 6 rejected 400 ORDER_LIMIT_EXCEEDED	Unit	M	Auto	Pass
TC-QR-01	FR-011	Generate pass for valid input: 201, 64-hex signature, PNG data URL, payload echoed	Unit + Integ	H	Auto	Pass
TC-QR-02	FR-013, SEC-006	Change eventId in a signed pass: INVALID_SIGNATURE	Unit	H	Auto	Pass
TC-QR-03	FR-013	Verify a pass whose expiresAt is in the past: EXPIRED_QR	Unit	H	Auto	Pass
TC-QR-04	FR-011	POST /api/qr/pass without ticketId: 400 INVALID_QR_REQUEST	Integ	H	Auto	Pass
TC-QR-05	FR-011	POST /api/qr/pass for a ticket whose stored expiry has passed: 400 EXPIRED_PASS	Integ	M	Auto	Pass
TC-QR-06	SEC-003	Verify QR data with missing fields: MISSING_QR_FIELDS	Unit	M	Auto	Pass
TC-SCN-01	FR-014	First scan of a valid pass: 200 CHECKED_IN with checkedInAt	Unit + Integ	H	Auto	Pass
TC-SCN-02	FR-014, BR-003	Second scan of the same pass: 409 DUPLICATE_ENTRY	Unit + Integ	H	Auto	Pass
TC-SCN-03	FR-013	Pass for event A scanned at event B: 400 WRONG_EVENT	Unit + Integ	H	Auto	Pass
TC-SCN-04	FR-013	Pass with altered signature at API: 400 INVALID_SIGNATURE	Integ	H	Auto	Pass
TC-SCN-05	SEC-003	Check-in with missing booking status fields: MISSING_CHECKIN_FIELDS	Unit	M	Auto	Pass
TC-WTL-01	FR-015	Join waitlist: entry created in FIFO order; second join by same user rejected 409	Integ	M	Auto	Pass
TC-WTL-02	FR-016	Seats released: first user in queue gets exactly one in-app notification and a 15-minute claim	Integ	M	Auto	Pass
TC-WTL-03	FR-016, BR-004	Offer not claimed in window (15 s in test): offer moves to second user	Integ	M	Auto	Pass
TC-ANL-01	FR-017	Scan a pass: telemetry snapshot and SSE stream reflect the new check-in	System	M	Auto	Pass
TC-ANL-02	FR-018	Export CSV: row count and revenue total match DB query	Integ	M	Auto	Pass
TC-ADM-01	FR-019	Admin approves event: state PUBLISHED; non-admin gets 403	Integ	H	Auto	Pass
TC-ADM-02	FR-019	Deactivated user's next request: 401	Integ	H	Auto	Pass
TC-PERF-01	NFR-001	500 concurrent users on catalog search: p95 ≤ 150 ms (16.2 ms with 1 s pause, DEV-007)	System	H	Auto	Pass

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-PERF-02	NFR-002	1,000 sequential valid scans: server p99 < 200 ms	System	H	Auto	Pass
TC-PERF-03	NFR-003, FR-010	200 parallel holds on a 1-seat tier: exactly 1 x 201, 199 x 409, quota never negative	System	H	Auto	Pass
TC-OPS-01	NFR-004	Health, live and ready endpoints and 60 s availability poller (TC-OPS-01.1 to 01.13)	System	M	Auto	Pass
TC-OPS-02	NFR-006	Coverage report ≥ 80% statements and branches	Unit	H	Auto	Pass
TC-OPS-03	NFR-007	Reconnect PostgreSQL: bookings, tickets, check-ins and audit rows still present; early deletion blocked	System	M	Auto	Pass
TC-UX-01	NFR-005	Five first-time users book a ticket; count screens; all ≤ 4	Accept	M	Man	Pass
TC-UX-02	FR-012	Attendee pass screen shows name, event, tier and a scannable QR fetched from GET /api/qr/pass/:ticketId	System	H	Man	Pass
TC-SEC-01	SEC-002	Role matrix: each protected route with each wrong role returns 403	Integ	H	Auto	Pass
TC-SEC-02	SEC-003	Injection strings in search and event fields return 400 or an empty list (see TC-SEC-04 to 07)	System	H	Auto	Pass
TC-SEC-03	SEC-004	TLS 1.2 handshake refused; http:// redirects to https://	System	H	Man	Pass
TC-SEC-AUDIT-01 to 12	SEC-005	Failed logins, rejected scans and admin actions each produce one audit row; audit rows cannot be updated or deleted (was TC-SEC-04)	Integ	M	Auto	Pass
TC-QR-SIG-05	SEC-006	QR_HMAC_SECRET unset: signing and verification fail closed with a configuration error (was TC-SEC-05)	Unit	M	Auto	Pass

5.3 Sample Detailed Test Case

Field	TC-SCN-02: Reject duplicate check-in
Module	Check-in Service / API
Requirement	FR-014, BR-003
Preconditions	Server running with QR_HMAC_SECRET set; check-in store cleared (`clearCheckInRecords()` in beforeEach)
Test data	ticketId T-1001, eventId E-42, userId U-7, expiresAt now + 1 hour; ticketStatus ACTIVE; bookingConfirmationStatus CONFIRMED
Steps	1. Generate a pass with generateQrPass(). 2. POST /api/checkin/verify with qrData and eventId E-42. 3. Repeat step 2 with the same body.
Expected	Step 2: 200, code CHECKED_IN. Step 3: 409, code DUPLICATE_ENTRY.
Actual	As expected (src/routes/checkin.routes.test.js); also passes against PostgreSQL with the unique index on check_in_logs
Result	Pass

5.4 Test Cases Added During the Sprints

TC IDs	Req	What they cover
TC-AUTH-06 to 09	FR-001, FR-003	Login edge cases and gate staff authentication
TC-AUTH-SEC-01 to 03	SEC-002	Role in the registration body is ignored (privilege escalation guard)
TC-PROV-01 to 05	FR-001, SEC-002	Admin provisioning of organizer, gate staff and admin accounts
TC-SEED-01, 02	SEC-001	Seed accounts hashed and isolated from production
TC-MID-01 to 05	FR-003	Auth middleware: missing, malformed, expired tokens and deactivated users

TC IDs	Req	What they cover
TC-SEC-04 to 07	SEC-003	SQL injection and XSS payloads
TC-QR-SIG-01 to 05	SEC-006	Signature tampering, key order, constant-time compare, missing secret
TC-SEC-AUDIT-01 to 12	SEC-005	Audit rows and immutability
TC-OPS-01.1 to 01.13	NFR-004	Health endpoints and availability poller
TC-PERF-02.1 to 02.4	NFR-002	Check-in latency, duplicate rejection latency, Server-Timing header, PostgreSQL path

ID mapping. TC-SEC-04 and TC-SEC-05 from version 1.0 were reused during Sprint 2 for injection tests. The original TC-SEC-04 (audit) is now TC-SEC-AUDIT-01 to 12 and the original TC-SEC-05 (missing secret) is now TC-QR-SIG-05. Full descriptions are in `docs/TEST_CASES_UPDATED.md`.

6. Test Environment

Item	Details
Hardware	Developer laptops; GitHub-hosted Ubuntu runners; phones with cameras for scanner and pass checks
Software	Node.js 20, PostgreSQL 16-alpine and Redis 7-alpine via Docker Compose or CI service containers, Nginx, Chrome / Firefox / Edge / Safari
Tools	Jest and Supertest (unit, integration), pg-mem, Node load benchmark and k6 script (performance), openssl and curl via check-tls.sh (TLS), oxlint (client static checks), GitHub Projects board for defects
CI	GitHub Actions on every push and pull request: server `npm test -- --coverage` with PostgreSQL and Redis services and an 80% gate; client `oxlint` and `vite build`
Test data	Seed data creates users for each role, event "TechSummit 2026" with tiers Early Bird (50 @ 199), General (300 @ 399), VIP (20 @ 799), plus one 1-seat tier for concurrency tests. No real personal data.

7. Test Schedule

Milestone	Timeline
Unit tests for QR and check-in modules	Completed before Sprint 1
Test case design (all modules)	Completed in Sprint 1
Integration testing per merged module	Completed in Sprints 1 and 2
System, performance and security testing	Completed in Sprint 2
Regression	Every push and pull request in CI
Usability walkthrough (TC-UX-01)	Completed before the final demo
Test summary (unit test, coverage and validation reports)	Completed at the end of Sprint 2

8. Test Deliverables

- Test Plan (this document)
- Test case catalogue (Section 5.2) and `docs/TEST_CASES_UPDATED.md`
- Jest/Supertest test files in `server/src/**/*.test.js` and `server/tests`; performance results in `docs/perf`
- Seed data script
- CI run logs and coverage reports (HTML from Jest)
- Unit test, code coverage, branch coverage and system validation reports in `docs/`
- Defect reports (GitHub issues labelled `bug`, with requirement ID in the title)
- Deviations log (`docs/DEVIATIONS.md`)

9. Roles and Responsibilities

Role	Responsibility
QA Lead	Owns this plan and the CI test stage; QR, check-in, telemetry and report tests
Security Test Engineer	Auth, RBAC and security validation (5.1)
Performance Test Engineer	Booking and waitlist tests, concurrency and retention runs
Product Owner / Test Engineer	Event module tests, search performance, TLS check, usability walkthrough, sign-off of acceptance
Developers (all)	Unit tests with each pull request; fix and retest assigned defects

10. Risks and Mitigation

Risk	Mitigation
Modules merged late leave little time for integration testing	Contracts in SAD 4.3 agreed early; tests written against stubs; integration starts per module as soon as it merges
Load tests on a laptop do not reflect a real server	Method and hardware recorded with each result in `docs/perf`; think-time assumption logged as DEV-007
Flaky tests due to hold expiry timing	Tests drive the expiry worker directly and set deadlines in the database instead of waiting
Local services unavailable	PostgreSQL and Redis suites also run in CI service containers
Same developer writes code and tests	Every pull request reviewed by a second member who also reviews the tests

11. Assumptions and Dependencies

- docker-compose brings up PostgreSQL 16 and Redis 7 on all team machines.
- Seed data is loaded before integration testing starts.
- At least two phones with working cameras are available for scanner testing.
- Five students outside the team agree to take part in the usability walkthrough.

12. Suspension and Resumption Criteria

Suspend testing if:

- The build fails to start or more than 30% of planned test cases in the cycle are blocked.
- A Critical defect in auth or booking is found (for example, an oversell or an RBAC bypass); dependent tests stop until fixed.
- The test environment is unavailable for more than 4 hours.

Resume testing when: the blocking defect is fixed and its test passes, a smoke test of the affected module passes, and the environment has been stable for one hour. Blocked test cases are re-run from the start.

13. Test Case Management and Traceability

Every test case carries the requirement ID it verifies, and every requirement has at least one test case. The full mapping and implementation status is the RTM in SRS Section 8. Test case IDs follow TC-<MODULE>-NN. Where a test does not carry the TC ID in its name, the suite file below is the evidence.

Requirement	Test cases	Evidence (suite)
FR-001 to FR-003	TC-AUTH-01 to 09, TC-MID-01 to 05	tests/auth.test.js, middleware tests
FR-004 to FR-007	TC-EVT-01 to 05, TC-TCK-01	tests/events.test.js, event_management.test.js, event_lifecycle.test.js, tiers.test.js

Requirement	Test cases	Evidence (suite)
FR-008 to FR-010, NFR-003	TC-BKG-01 to 04, TC-PERF-03	src/services/booking-hold.postgres.test.js
FR-011, FR-012	TC-QR-01 to 06, TC-UX-02	qr.service.test.js, qr.routes.test.js, attendeePass.routes.test.js
FR-013, FR-014	TC-SCN-01 to 05	checkin.service.test.js, checkin.routes.test.js
FR-015, FR-016	TC-WTL-01 to 03	src/services/waitlist.postgres.test.js
FR-017, FR-018	TC-ANL-01, TC-ANL-02	telemetry and report service and route tests, csv.util and pdf.util tests
FR-019	TC-ADM-01, TC-ADM-02	tests/admin.test.js
NFR-001, NFR-002	TC-PERF-01, TC-PERF-02.1 to 02.4	tests/perf/search.perf.test.js, checkin.perf.test.js
NFR-004, NFR-007	TC-OPS-01.1 to 01.13, TC-OPS-03	health and availability tests, tests/nfr-007-retention.postgres.test.js
SEC-001 to SEC-006	See 5.1	See 5.1

14. Test Metrics and Reporting

Metrics collected: % test cases executed; % passed and failed; defect density (defects per KLOC); defect ageing (days open by severity); requirement coverage (% of requirements with at least one passed test); code coverage (statements, branches).

Execution status at the end of Sprint 2

Measure	Value
Test cases in catalogue (Section 5.2)	46
Executed / passed / failed	46 / 46 / 0
Automated Jest suites and tests	36 suites, 496 tests, all passing
Statement / branch / line coverage	92.42% / 82.17% / 93.67%
Requirements with a passed test	31 of 32; FR-015 partly met (sold-out check on waitlist join)
Repository issues	43 of 44 closed
Open defects	0 Critical, 0 High; open items OI-4 (CORS) and OI-7 (scanning staff ID not stored) are Minor

Reports: CI posts test and coverage results on each pull request. The unit test, code coverage, branch coverage and system validation reports in `docs/` form the test summary.